

Bit Manipulation & Subsets

CodeIIEST Summer Bootcamp 2026

Week 05 · Prerequisites

Contents

1	Introduction	2
2	Various Bit Operations	2
2.1	Essential Bitwise Tricks	2
2.2	Built-in GCC Functions	2
3	Bitmasking (Subsets) & Submask Iteration	3
3.1	Generating All Subsets	3
3.2	Submask Iteration	3
4	Binary Exponentiation	4
5	Per Bit Contribution	5
6	XOR Properties & Prefix XOR	6
6.1	Fundamental Properties of XOR	6
6.2	Prefix XOR	6
7	Recommended Resources	6

1. Introduction

Everything in a computer is stored as 0s and 1s. *Bit manipulation* is the act of algorithmically manipulating these individual bits. Because bitwise operations are executed directly by the processor, they are incredibly fast ($O(1)$) and often hold the key to solving complex combinatorics, subset problems, and mathematical puzzles in competitive programming.

This week covers four vital concepts: Core Bit Operations, Bitmasking (Subsets), Per-Bit Contribution, and XOR Properties.

2. Various Bit Operations

Before utilizing advanced tricks, you must be entirely comfortable with the basic bitwise operators: $\&$ (AND), $\mid$ (OR), $\wedge$ (XOR), $\sim$ (NOT), $\ll$ (Left Shift), and $\gg$ (Right Shift).

2.1 Essential Bitwise Tricks

We zero-index bits starting from the right (the least significant bit is the 0-th bit).

- **Check if the i -th bit is set:** $(n \& (1 \ll i)) \neq 0$
- **Set the i -th bit** (turn it to 1): $n \mid (1 \ll i)$
- **Clear the i -th bit** (turn it to 0): $n \& \sim(1 \ll i)$
- **Toggle the i -th bit** (flip it): $n \wedge (1 \ll i)$
- **Remove the lowest set bit:** $n \& (n - 1)$ (Very common trick!)
- **Extract the lowest set bit:** $n \& -n$
- **Multiply by 2^k :** $n \ll k$
- **Divide by 2^k :** $n \gg k$

Crucial Avoidance of Overflow: In C++, the literal 1 is a 32-bit signed integer. If you try to shift it by 31 or more (e.g., $1 \ll 40$), it will cause an overflow and give garbage results. Always use `1LL << i` when working with 64-bit integers (`long long`).

2.2 Built-in GCC Functions

GCC provides fast built-in functions to analyze bits in $O(1)$ time.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int n = 10; // Binary: 1010
6
7     // 1. Count number of set bits (1s)
8     int count = __builtin_popcount(n); // Returns 2
9     long long num = 1e15;
10    int countLL = __builtin_popcountll(num); // Use 'll' suffix for
        long long
11
12    // 2. Count leading zeros
13    int lz = __builtin_clz(n);
14
15    // 3. Count trailing zeros
```

```

16     int tz = __builtin_ctz(n); // Returns 1 (because 1010 has one zero
17     at the end)
18     return 0;
19 }

```

3. Bitmasking (Subsets) & Submask Iteration

Concept

A **Bitmask** is simply an integer where we treat its binary representation as a boolean array. This allows us to represent a subset of a set using a single number.

If we have N items, there are 2^N possible subsets. We can map each subset to an integer from 0 to $2^N - 1$. For example, if $N = 3$ items $\{A, B, C\}$, the integer 5 (Binary 101) means items A and C are included, but B is not.

3.1 Generating All Subsets

This allows us to write elegant brute-force solutions for $N \leq 20$ in $O(N \cdot 2^N)$ time.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void generateSubsets(vector<int>& arr) {
5      int n = arr.size();
6      int totalSubsets = 1 << n; // 2^n
7
8      // Iterate through all possible masks
9      for (int mask = 0; mask < totalSubsets; mask++) {
10         vector<int> subset;
11         // Check which items are present in the current mask
12         for (int i = 0; i < n; i++) {
13             if ((mask & (1 << i)) != 0) {
14                 subset.push_back(arr[i]);
15             }
16         }
17
18         // Print the subset
19         cout << "{ ";
20         for (int x : subset) cout << x << " ";
21         cout << "}\n";
22     }
23 }

```

3.2 Submask Iteration

Sometimes, for a given subset (`mask`), you need to iterate over all of its sub-subsets (sub-masks). A naive approach would check all 2^N masks, but there is an elegant trick to do this efficiently.

```

1 int mask = 26; // Binary: 11010 (Bits 1, 3, 4 are set)
2
3 // Iterates exactly over the submasks of 'mask'
4 // (e.g., 11010, 11000, 10010, 01010, 10000...)
5 for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
6     // Process submask 'sub'
7 }
8 // Note: To include the empty subset (0), process 'sub = 0' separately
   // after the loop.

```

Time Complexity Magic: If you iterate over all 2^N masks, and for each mask, iterate over all of its submasks, the total time complexity is $O(3^N)$, not $O(4^N)$. This is heavily used in advanced Dynamic Programming with Bitmasking.

4. Binary Exponentiation

Computing a^b by multiplying a exactly b times is $O(b)$ — far too slow when b can be up to 10^{18} . *Binary exponentiation* (also called *fast power* or *exponentiation by squaring*) drops this to $O(\log b)$ by exploiting the binary representation of the exponent — the same bit-decomposition idea behind subset enumeration, applied to the bits of b .

Key Idea

Write b in binary. Then a^b is the product of a^{2^k} over exactly those bits k that are set in b . We walk the bits of b from the lowest, repeatedly squaring a to get $a^1, a^2, a^4, a^8, \dots$, and fold a factor into the answer only when the current bit is set.

For example, a^{13} : since $13 = (1101)_2 = 8 + 4 + 1$, we have $a^{13} = a^8 \cdot a^4 \cdot a^1$ — three multiplications instead of twelve.

In practice you almost always want the result modulo a prime (typically $10^9 + 7$), since raw powers overflow instantly. Keep every intermediate product reduced.

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const long long MOD = 1e9 + 7;
5
6 // Computes (a^b) % MOD in O(log b)
7 long long binpow(long long a, long long b, long long mod) {
8     a %= mod; // guard against a ≥ mod
9     long long result = 1;
10    while (b > 0) {
11        // If the lowest bit of b is set, fold a into the result
12        if (b & 1) result = (result * a) % mod;
13        a = (a * a) % mod; // square the base for the next bit
14        b >>= 1; // move to the next bit of the exponent
15    }
16    return result;
17 }

```

Where it shows up: the squaring trick is not limited to integers. The exact same skele-

ton works for anything associative — matrix powers (linear recurrences like Fibonacci in $O(\log n)$), modular inverses via Fermat's little theorem (`binpow(a, MOD - 2, MOD)` when MOD is prime), and even fast multiplication via *binary multiplication* when the product itself would overflow 64 bits.

Overflow caveat: `result * a` with two values near MOD ($\approx 10^9$) fits in `long long` ($\approx 9.2 \times 10^{18}$), so this is safe for $\text{MOD} \leq 3 \times 10^9$ or so. If your modulus is larger, switch to `__int128` for the multiplication or use binary multiplication.

5. Per Bit Contribution

Concept

When asked to find the sum of a bitwise operation (AND, OR, XOR) over all pairs or subsets of an array, a brute force $O(N^2)$ approach will fail. Instead, isolate the problem **bitwise**. Solve for the 0-th bit, then the 1st bit, up to the 31st bit independently, and sum their contributions.

Classic Problem: Find the sum of XOR of all pairs in an array.

$$\sum_{i=1}^N \sum_{j=i+1}^N (A_i \oplus A_j)$$

Logic: Look at just the i -th bit across all numbers. An XOR pair ($A_x \oplus A_y$) will contribute a 1 to the i -th bit of the total sum only if one number has a 0 at the i -th bit and the other has a 1. So, count the total numbers with a 0 at the i -th bit (`count_0`) and the total numbers with a 1 (`count_1`). The number of pairs that yield a 1 is exactly $\text{count}_0 \times \text{count}_1$. Because this is the i -th bit, its actual numerical value contribution to the sum is $(\text{count}_0 \times \text{count}_1) \times 2^i$.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 long long sumPairXOR(vector<int>& arr) {
5     long long total_sum = 0;
6     int n = arr.size();
7
8     // Check bits from 0 to 31
9     for (int i = 0; i < 32; i++) {
10         long long count_1 = 0, count_0 = 0;
11
12         for (int j = 0; j < n; j++) {
13             if (arr[j] & (1 << i)) count_1++;
14             else count_0++;
15         }
16
17         // Number of pairs that give '1' at this bit
18         long long valid_pairs = count_1 * count_0;
19
20         // Add contribution to the total sum
21         total_sum += valid_pairs * (1LL << i);
22     }
23     return total_sum;
24 }
```

6. XOR Properties & Prefix XOR

XOR ($\oplus$) is the most frequently tested bitwise operator because of its unique mathematical properties.

6.1 Fundamental Properties of XOR

- $x \oplus x = 0$ (A number XORed with itself cancels out).
- $x \oplus 0 = x$ (0 is the identity element).
- $x \oplus y = y \oplus x$ (Commutative).
- $(x \oplus y) \oplus z = x \oplus (y \oplus z)$ (Associative).
- If $a \oplus b = c$, then $a \oplus c = b$ and $b \oplus c = a$.

6.2 Prefix XOR

Just like Prefix Sums, we can compute Prefix XORs to answer range queries in $O(1)$ time. This works precisely because of the cancellation property ($x \oplus x = 0$).

If `pref[i]` stores the XOR of elements from index 0 to i , then the XOR of elements in the range $[L, R]$ is:

$$\text{Query}(L, R) = \text{pref}[R] \oplus \text{pref}[L - 1]$$

(Assuming 1-based indexing).

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      vector<int> a = {3, 1, 4, 6, 2}; // 1-based indices: 1 to 5
6      int n = a.size();
7      vector<int> pref(n + 1, 0);
8
9      // Precompute prefix XORs: O(N)
10     for (int i = 1; i ≤ n; i++) {
11         pref[i] = pref[i - 1] ^ a[i - 1];
12     }
13
14     // Query XOR from index L=2 to R=4 (Values: 1, 4, 6)
15     int L = 2, R = 4;
16     int range_xor = pref[R] ^ pref[L - 1];
17
18     // 1 ^ 4 ^ 6 = 3. Output will be 3.
19     cout << "Range XOR is: " << range_xor << "\n";
20     return 0;
21 }
```

7. Recommended Resources

Bit manipulation is heavily driven by practice. It requires you to observe binary patterns on paper. Dive into these resources to master the techniques:

- **USACO Guide:** Excellent curated problems and deeper theory.

- [Silver: Bitwise Operators](#)
- **CP-Algorithms:** The bible for competitive programming math.
 - [Bit Manipulation Techniques](#)
 - [Binary Exponentiation](#)
- **YouKnowWho Academy (CodeForces ICPC-style topic list):** structured ladders with curated problems.
 - [Bit Manipulation](#)
 - [Modular Arithmetic](#)
- **YouTube Channels:**
 - **Errichto:** Bitwise Operations for Beginners (A must-watch logic breakdown).
 - **Luv:** Bit Manipulation Playlist (Specifically good for generating subsets and basic bit masking).